Docker - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

DOCKER — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Written for someone who has never used Docker before — covers local usage, company/team usage, and large-scale production usage, with diagrams

---

TOPIC 1: IMAGES

---

1. Introduction

1.1 What is Docker, and why does it exist?

Docker is a platform for packaging an application TOGETHER WITH everything it needs to run (code, runtime, system libraries, configuration) into a single, portable unit called a **container** — so it runs IDENTICALLY on your laptop, a teammate's laptop, a test server, and production, regardless of what's actually installed on each of those machines.

1.2 The EXACT problem Docker solves — "It works on my machine"

Every developer has lived this:

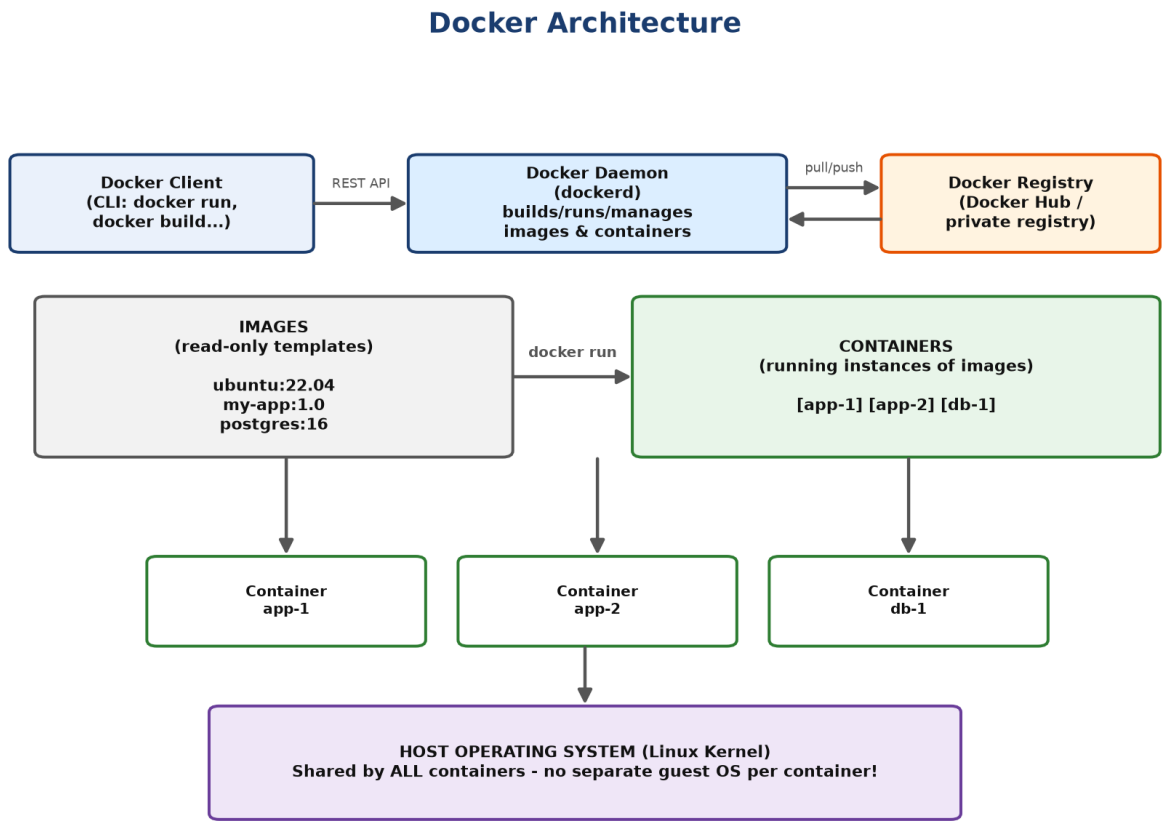
```
1. You build a Java app on YOUR laptop (Java 17, specific library versions, a locally
running PostgreSQL database on port 5432) - it works perfectly.
2. You hand it to a TEAMMATE, or deploy it to a SERVER.
3. That machine has Java 11 instead of 17, a different PostgreSQL version, a
missing environment variable, a different OS - and the app BREAKS, or behaves
subtly differently.
4. Hours are lost debugging "environment differences" instead of actual code bugs.
```

Docker eliminates this entirely — you package your app and its EXACT environment (the correct Java version, exact library versions, configuration) into ONE image. That SAME image runs identically EVERYWHERE, because Docker doesn't rely on whatever happens to already be installed on the host machine.

1.3 What is a Docker Image?

An **image** is a READ-ONLY, portable TEMPLATE containing everything needed to run an application — application code, a runtime (like the JRE), system tools, libraries, and configuration. Images are BUILT ONCE (from a `Dockerfile`, Topic 3) and then used to CREATE containers (running instances of that template) as many times as needed.

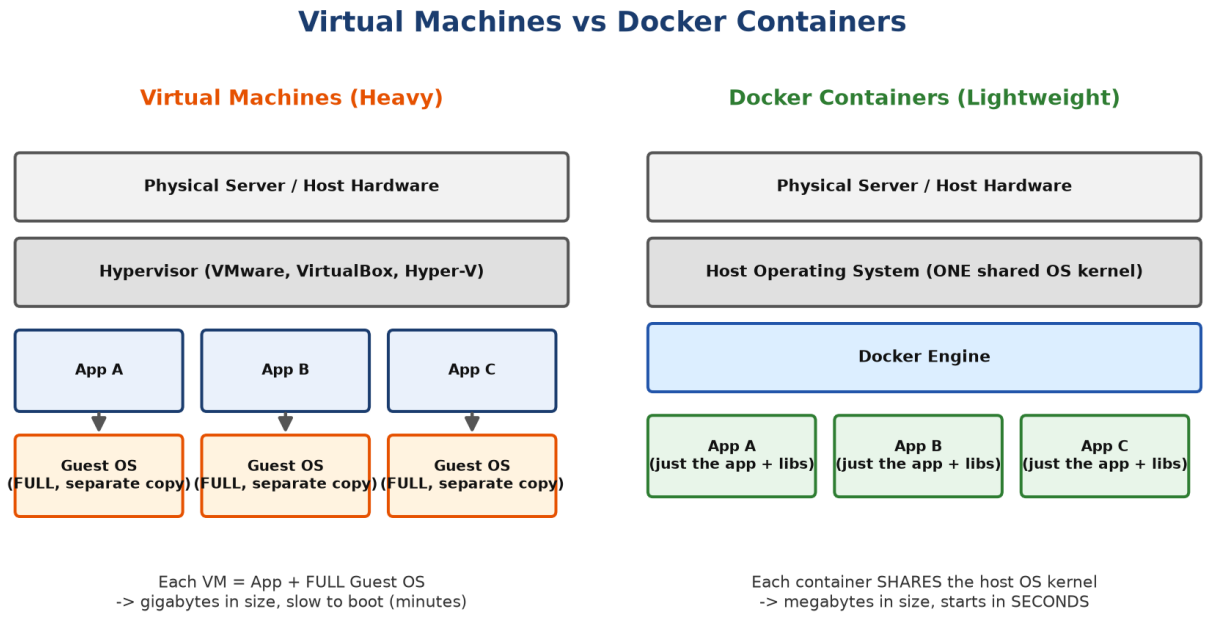
2. Docker Architecture — The Big Picture



Docker Architecture: Client, Daemon, Images, Containers, and Registry

- The **Docker Client** is the command-line tool (`docker`) YOU type commands into.
- The **Docker Daemon** (`dockerd`) is the background service that actually builds images and runs containers.
- **Images** are the read-only templates (`postgres:16`, `my-app:1.0`).
- **Containers** are running instances CREATED FROM images.
- The **Registry** (Docker Hub, or a private company registry) is where images are STORED and SHARED — like GitHub, but for images instead of code.

3. Why Containers, Not Virtual Machines? (A Critical Distinction)



Virtual Machines vs Docker Containers — size and startup time comparison

Aspect	Virtual Machine	Docker Container
What's included	A FULL, separate guest Operating System per app	Just the app + its specific libraries — SHARES the host's OS kernel
Size	Gigabytes	Megabytes
Startup time	Minutes	Seconds (often sub-second)
Isolation level	Very strong (fully separate kernel)	Strong, but lighter-weight (shared kernel, isolated via Linux namespaces/cgroups)
Resource overhead	High (each VM duplicates an entire OS)	Low (no OS duplication)

This is WHY Docker exploded in popularity for modern application deployment — you can run DOZENS of containers on a single server with the SAME hardware that might only comfortably run a handful of full VMs.

4. Key Image Commands

```
docker images # list all images currently downloaded/built on your machine
docker pull postgres:16 # download a specific image FROM Docker Hub (a public registry)
docker rmi postgres:16 # remove (delete) a local image
docker image inspect postgres:16 # view detailed metadata about an image (layers, config, environment)
docker history my-app:1.0 # see each LAYER that makes up an image and its size
```

5. Image Naming — Tags

```
postgres:16
| |
| +-- TAG (a specific version; "latest" is the default if omitted)
+-------- REPOSITORY NAME (which image)

my-company-registry.com/team-payments/order-service:1.4.2
| | | |
registry hostname namespace/team image name version tag
```

6. Real Life Analogy

- **Restaurant**: A Docker image is like a COMPLETE, standardized, pre-packaged meal-kit recipe box — containing EXACTLY the right, pre-measured ingredients and step-by-step instructions. Any kitchen (any machine), anywhere in the world, that opens this EXACT SAME meal kit produces the EXACT SAME dish, regardless of what that kitchen already had in its own pantry.

7. Edge Cases

Case	Result
Pulling an image without specifying a tag (e.g., `docker pull ubuntu`)	Defaults to the `latest` tag — often NOT recommended for production, since "latest" can silently change over time
Trying to delete an image currently used by a running container	Fails — you must stop/remove the container first

8. Common Mistakes

- Using the `latest` tag in production configurations — it's a MOVING TARGET; always pin an EXACT version (e.g., `postgres:16.1`) for reproducible, predictable deployments.
- Not understanding that images are IMMUTABLE — you don't "edit" a running container's image; you build a NEW image version and deploy that instead.

9. Frequently Asked Interview Questions

1. **What problem does Docker solve?** The "it works on my machine" problem — packaging an application with its exact environment so it runs identically everywhere, regardless of what's installed on the host machine.
2. **What is a Docker image?** A read-only, portable template containing everything needed to run an application (code, runtime, libraries, config).
3. **What's the key architectural difference between a VM and a container?** A VM includes a full, separate guest OS per application; a container shares the host's OS kernel, making it dramatically smaller and faster to start.
4. **Why is using the `latest` tag in production discouraged?** It's a moving target — the actual image content behind "latest" can change over time, breaking reproducibility; always pin an exact version tag.

TOPIC 2: CONTAINERS

1. Introduction

What is it? A **container** is a RUNNING (or stopped) INSTANCE of an image — the actual, live, isolated process executing your application. You can create MANY containers from the SAME image, each running independently.

2. FULL STEPS: Installing Docker and Running Your First Container Locally

STEP 1 — Install Docker Desktop (Windows/Mac) or Docker Engine (Linux)

1. Go to **https://www.docker.com/products/docker-desktop/** and download **Docker Desktop** for your OS (Windows or Mac).
2. Run the installer, following the prompts (on Windows, this requires **WSL2** — Windows Subsystem for Linux — which the installer will help enable/configure if not already present).
3. Launch Docker Desktop and wait for it to show "Docker is running" (a whale icon in your system tray/menu bar).

STEP 2 — Verify the Installation

```
docker --version
docker run hello-world
```

The `hello-world` command downloads a tiny test image and runs it, printing a confirmation message — if you see it, Docker is correctly installed and working.

STEP 3 — Run a Real Container

```
docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=mysecret --name my-postgres postgres:16
```

Flag	Meaning
`-d`	Detached mode — run in the BACKGROUND, not tied to your terminal
`-p 5432:5432`	Port mapping: `host_port:container_port` — makes the container's port 5432 reachable at `localhost:5432` on your machine
`-e POSTGRES_PASSWORD=mysecret`	Sets an environment variable INSIDE the container
`--name my-postgres`	Gives the container a memorable name (instead of a random auto-generated one)
`postgres:16`	The image to run

STEP 4 — Manage the Running Container

```
docker ps # list RUNNING containers
docker ps -a # list ALL containers, including stopped ones
docker logs my-postgres # view a container's console output/logs
docker exec -it my-postgres bash # open an interactive shell INSIDE the running container
docker stop my-postgres # gracefully stop it
docker start my-postgres # start it again (reuses the SAME container)
docker rm my-postgres # permanently remove the (stopped) container
```

3. Container Lifecycle

```
docker run -> CREATED -> RUNNING -> (docker stop) -> STOPPED -> (docker rm) -> REMOVED
|
(docker pause) -> PAUSED -> (docker unpause) -> RUNNING
```

4. Real Life Analogy

- **Restaurant**: If the Image is the meal-kit recipe box, a Container is the ACTUAL DISH being cooked RIGHT NOW from that box — you can make MANY separate dishes (containers) from the SAME recipe box (image) simultaneously, each one independent, and each one can be started, paused, or thrown away without affecting the original recipe box itself.

5. Edge Cases

Case	Result
Running the SAME `docker run` command twice with the SAME `--name`	Fails — container names must be UNIQUE; remove the old one first, or omit `--name` for an auto-generated unique one
Stopping a container	Its DATA, by default, is LOST when the container is later removed, UNLESS you're using volumes (Topic 5)
Forgetting `-d` (detached mode)	The container runs in the FOREGROUND, tying up your terminal until you `Ctrl+C` (which typically also stops the container)

6. Common Mistakes

- Forgetting that container data is EPHEMERAL by default — restarting/removing a container without a volume WIPES any data written inside it.
- Confusing `docker stop` (graceful, keeps the container for later restart) with `docker rm` (permanently deletes the container).

7. Frequently Asked Interview Questions

1 **What is a container?** A running (or stopped) instance of a Docker image — an isolated, executing process containing your application.

2 **What does the `-p 8080:80` flag do in `docker run`?** Maps port 80 INSIDE the container to port 8080 on the HOST machine, making the containerized service reachable at `localhost:8080`.

3 **What's the difference between `docker stop` and `docker rm`?** `stop` gracefully halts a running container (which can be restarted later with `docker start`); `rm` permanently deletes the container entirely.

4 **Is a container's data persisted by default if the container is removed?** No — data is ephemeral unless explicitly persisted via a volume (Topic 5).

TOPIC 3: DOCKERFILE

---

1. Introduction

What is it? A `Dockerfile` is a plain TEXT file containing step-by-step INSTRUCTIONS for building a Docker image — think of it as a "recipe" that Docker reads and executes to produce your image, so anyone (or any CI/CD pipeline) can rebuild the EXACT same image reproducibly.

2. FULL STEP-BY-STEP: Dockerizing a Spring Boot Application

STEP 1 — Write the Dockerfile

```
# Dockerfile (place this in your project's root directory, alongside pom.xml)

FROM eclipse-temurin:17-jre-alpine # base image: a minimal Linux + Java 17 runtime

WORKDIR /app # sets the working directory INSIDE the container

COPY target/employee-api-1.0.0.jar app.jar # copies your BUILT jar (from Maven's target/) into the image

EXPOSE 8080 # documents which port the app listens on (informational)

ENTRYPOINT ["java", "-jar", "app.jar"] # the command that runs when a container STARTS
```

STEP 2 — Build the Jar First (Maven, from earlier in this course)

```
mvn clean package # produces target/employee-api-1.0.0.jar
```

STEP 3 — Build the Docker Image

```
docker build -t employee-api:1.0.0 .
```

Part	Meaning
`docker build`	Build a new image
`-t employee-api:1.0.0`	TAG (name + version) the resulting image
`.`	Build CONTEXT — the current directory, where Docker looks for the `Dockerfile` and any files you `COPY`

STEP 4 — Run YOUR Application as a Container

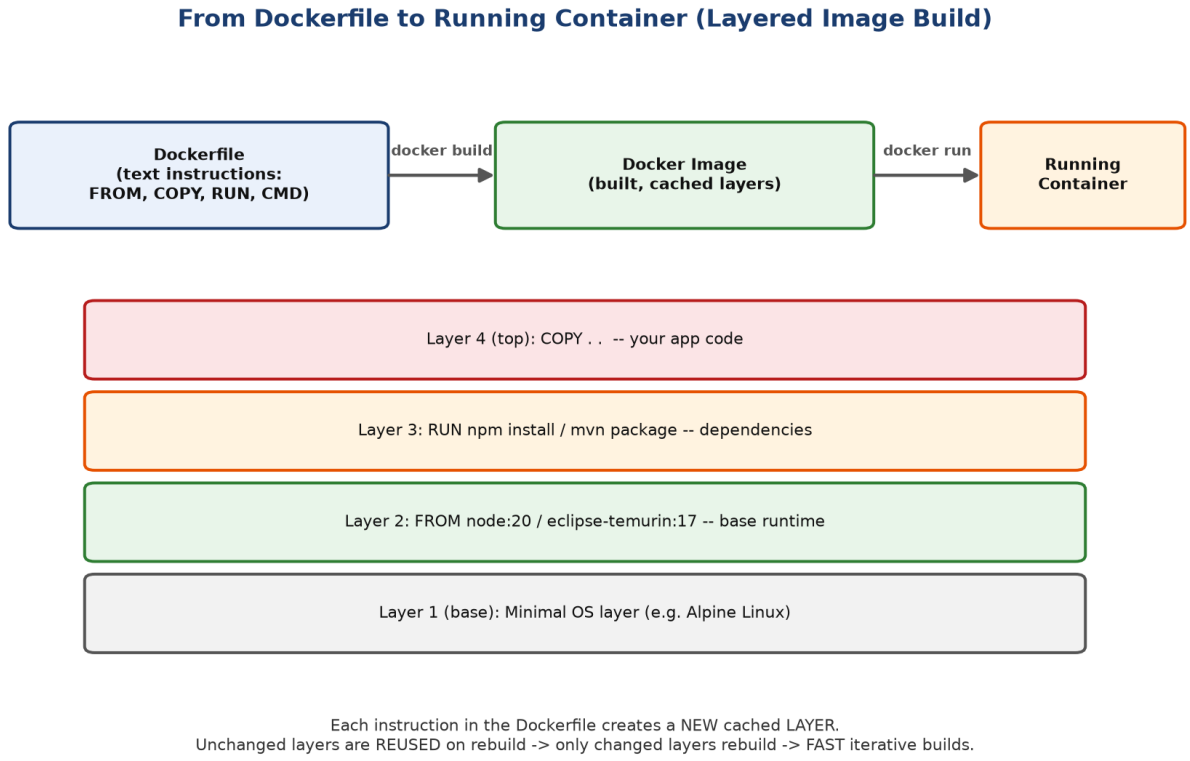
```
docker run -d -p 8080:8080 --name my-employee-api employee-api:1.0.0
```

Your Spring Boot app is now running INSIDE a container, reachable at `http://localhost:8080` — exactly as if you'd run it directly, but now fully packaged and portable.

3. Key Dockerfile Instructions

Instruction	Purpose
`FROM`	Sets the BASE image to build on top of (almost always the first line)
`WORKDIR`	Sets the working directory for subsequent instructions
`COPY`	Copies files from your build context INTO the image
`RUN`	Executes a command DURING the build (e.g., installing packages, compiling)
`EXPOSE`	Documents which port the container listens on (doesn't actually publish it — that's `docker run -p`)
`ENV`	Sets an environment variable inside the image
`ENTRYPOINT` / `CMD`	Defines what command runs when a container STARTS

4. How Layers and Caching Work (Full Detail)



From Dockerfile to Running Container — layered image build

Each instruction creates a new, CACHED layer. If you change ONLY your application code (the `COPY` line), Docker REUSES all the earlier, unchanged layers (the base image, dependencies) and only rebuilds the changed layer — making REPEATED builds during development MUCH faster than rebuilding everything from scratch every time.

> **Practical tip**: Order your Dockerfile instructions from LEAST-frequently-changing (base image, dependency installation) to MOST-frequently-changing (your actual application code) — this maximizes how often Docker can reuse cached layers.

5. Real Life Analogy

- **Restaurant**: A Dockerfile is like a WRITTEN, standardized recipe card — "start with THIS base stock, add THESE specific ingredients in THIS order, cook for THIS long." Anyone (any kitchen, any CI/CD pipeline) following that EXACT recipe card produces an IDENTICAL final dish (image), every single time.

6. Edge Cases

Case	Result
Changing a line NEAR THE TOP of the Dockerfile (e.g., the base image version)	INVALIDATES the cache for EVERY layer below it — forces a full rebuild from that point downward
Forgetting `EXPOSE`/`-p` port mapping	The container runs fine internally, but isn't reachable from OUTSIDE the container at all

7. Common Mistakes

- Placing frequently-changing instructions (like `COPY . .` for your entire source tree) too EARLY in the Dockerfile, defeating layer caching for everything below it.
- Forgetting to actually PUBLISH the port with `-p` when running (`EXPOSE` alone is only documentation, not a functional port mapping).

8. Frequently Asked Interview Questions

1. **What is a Dockerfile?** A text file containing step-by-step instructions for building a Docker image reproducibly.
2. **What's the difference between `EXPOSE` and the `-p` flag on `docker run`?** `EXPOSE` is DOCUMENTATION inside the Dockerfile (doesn't publish anything by itself); `-p` on `docker run` actually PUBLISHES/maps a container port to a host port.
3. **Why does Dockerfile instruction ORDER matter for build speed?** Docker caches each layer; changing an EARLY instruction invalidates the cache for every layer after it — ordering from least-to-most frequently changing maximizes cache reuse.
4. **What command builds an image from a Dockerfile?** `docker build -t <name>:<tag> <build-context-path>`.

TOPIC 4: DOCKER COMPOSE

1. Introduction

What is it? Docker Compose lets you define and run MULTIPLE related containers TOGETHER (e.g., your Spring Boot app + a PostgreSQL database + Redis) using ONE YAML configuration file and a SINGLE command — instead of manually running several separate, long `docker run` commands with matching network/naming configuration every time.

2. FULL STEP-BY-STEP: Running a Full Local Development Stack

STEP 1 — Write `docker-compose.yml`

```
version: "3.8"

services:
app:
build: . # builds from the Dockerfile in the current directory
ports:
- "8080:8080"
environment:
- SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/company_db # 'db' = the OTHER service's name!
- SPRING_DATASOURCE_USERNAME=postgres
- SPRING_DATASOURCE_PASSWORD=mysecret
depends_on:
- db

db:
image: postgres:16
environment:
- POSTGRES_DB=company_db
- POSTGRES_PASSWORD=mysecret
ports:
- "5432:5432"
volumes:
- pgdata:/var/lib/postgresql/data # persists database data (Topic 5!)

redis:
image: redis:7-alpine
ports:
- "6379:6379"

volumes:
pgdata:
```

STEP 2 — Start EVERYTHING With ONE Command

```
docker compose up -d # builds (if needed) and starts ALL services defined above, in the background
```

STEP 3 — Manage the Stack

```
docker compose ps # see the status of all services
docker compose logs app # view logs for a SPECIFIC service
docker compose down # stop AND remove all containers (add -v to also remove volumes/data)
docker compose restart app # restart just one service
```

3. Why Compose Matters — The "Before" Comparison

```
# WITHOUT Compose - manually running 3 containers, matching network settings yourself:
docker network create my-app-net
docker run -d --network my-app-net --name db -e POSTGRES_PASSWORD=mysecret postgres:16
docker run -d --network my-app-net --name redis redis:7-alpine
docker run -d --network my-app-net --name app -p 8080:8080 -e
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/company_db my-app:1.0

# WITH Compose - ONE command, config lives in a SINGLE readable file:
docker compose up -d
```

4. Real Life Analogy

- **Restaurant**: Docker Compose is like a restaurant's ENTIRE meal SETUP INSTRUCTIONS — "the main dish, the side dish, and the drink ALL need to be prepared and served TOGETHER, in coordination" — instead of separately instructing three different, uncoordinated stations and hoping they line up correctly on their own.

5. Edge Cases

Case	Result
`app` service starting BEFORE `db` is actually ready to accept connections (despite `depends_on`)	`depends_on` only waits for the container to START, NOT for the database to be genuinely READY — a common real gotcha requiring a proper healthcheck or retry logic in the app itself
Running `docker compose down` without `-v`	Containers are removed, but named VOLUMES (like `pgdata`) PERSIST — your database data survives

6. Common Mistakes

- Assuming `depends_on` guarantees the dependency is FULLY READY (not just started) — use Compose `healthcheck` configurations or application-level retry logic for genuine readiness.
- Running `docker compose down -v` accidentally, wiping out persisted development database data.

7. Frequently Asked Interview Questions

1. **What problem does Docker Compose solve?** Running and coordinating MULTIPLE related containers together via one YAML file and one command, instead of manually running/networking each container separately.
2. **How do services in the same Compose file reach each other?** By SERVICE NAME (e.g., `db`) acting as a hostname on Compose's automatically-created shared network — not `localhost`.
3. **Does `depends_on` guarantee a dependency service is fully READY, not just started?** No — it only waits for the container to START; genuine readiness (e.g., the database accepting connections) requires healthchecks or retry logic.
4. **What's the difference between `docker compose down` and `docker compose down -v`?** The former removes containers but keeps named volumes (persisted data); the latter ALSO removes volumes, deleting that data.

TOPIC 5: VOLUMES

1. Introduction

What is it? A volume is Docker's mechanism for PERSISTING data BEYOND a container's own lifecycle — since containers are DESIGNED to be disposable (Topic 2), any data written INSIDE a container's own filesystem is LOST when that container is removed, UNLESS it's stored in a volume instead.

2. Code Examples — The Three Ways to Persist/Share Data

```
# NAMED VOLUME (Docker manages WHERE this data actually lives on the host - RECOMMENDED for real data)
docker run -d -v pgdata:/var/lib/postgresql/data postgres:16
docker volume ls # list all named volumes
docker volume inspect pgdata # see WHERE it's actually stored on the host

# BIND MOUNT (YOU specify the exact host path - great for LOCAL DEVELOPMENT, live-editing code)
docker run -d -v /home/user/my-project:/app my-app:1.0

# ANONYMOUS VOLUME (Docker creates one automatically, no name - rarely used deliberately)
docker run -d -v /var/lib/postgresql/data postgres:16
```

3. Named Volume vs Bind Mount — Comparison Table

Aspect	Named Volume	Bind Mount
Location control	Docker manages it internally	YOU specify the exact host directory
Best for	Production data (databases) — portable, Docker-managed	Local development — live-editing source code, seeing changes instantly
Portability	Works consistently across different host machines	Tied to that specific host's exact file layout

4. Real Life Analogy

- **Restaurant**: A container's own internal storage is like writing notes on a DISPOSABLE napkin — thrown away the moment that particular meal service ends. A volume is like writing notes in a PERMANENT, external ledger book kept SEPARATE from any single meal service — even if this evening's kitchen shift (container) ends, the ledger (data) survives for the NEXT shift to pick up exactly where it left off.

5. Frequently Asked Interview Questions

1 **Why are Docker volumes needed, given containers can already write files?** Because containers are DESIGNED to be disposable — any data written to a container's own filesystem is lost when it's removed; volumes persist data independently of any specific container's lifecycle.

2. **What's the difference between a named volume and a bind mount?** A named volume is managed internally by Docker (portable, ideal for production data); a bind mount maps to an EXACT host directory you specify (ideal for local development, live code editing).
3. **If you remove a container that was using a named volume, is the data lost?** No — the volume persists independently and can be reattached to a new container.

TOPIC 6: NETWORKS

---

1. Introduction

What is it? Docker Networks control HOW containers communicate with each other and with the outside world — by default, Docker creates isolated networks so containers can reach each other by SERVICE/CONTAINER NAME (not IP address), while remaining isolated from unrelated containers.

2. Code Examples

```
docker network ls # list all networks
docker network create my-app-network # create a custom network
docker run -d --network my-app-network --name db postgres:16
docker run -d --network my-app-network --name app my-app:1.0

# Inside the 'app' container, it can reach the database simply via the hostname "db" -
# Docker's internal DNS resolves container/service names automatically within a shared network
```

3. The Default Network Types

Type	Behavior
`bridge` (default)	Containers on the SAME bridge network can reach each other by name; isolated from containers on OTHER networks
`host`	Container shares the HOST's network directly (no isolation) — rarely used, mainly for specific performance/legacy needs
`none`	Completely disables networking for that container

4. Real Life Analogy

- **Bank**: A Docker network is like a bank's SECURE internal phone extension system — any department (container) connected to that SAME internal system can dial another department DIRECTLY by NAME/extension ("Loans Department," "Teller Station 3"), while completely UNRELATED, external phone lines (containers on a different network) simply can't reach into that internal system at all.

5. Frequently Asked Interview Questions

1. **How do containers on the same Docker network communicate with each other?** By CONTAINER/SERVICE NAME, resolved via Docker's built-in internal DNS — not by IP address or `localhost`.
2. **What's the default network type, and what does it provide?** `bridge` — containers on the same bridge network can reach each other by name, while remaining isolated from containers on other networks.
3. **Why can't a container reach another container using `localhost`?** Because each container has its OWN isolated network namespace — `localhost` INSIDE a container refers to that container itself, not the host machine or other containers.

TOPIC 7: MULTI-STAGE BUILDS

---

1. Introduction

What is it? A multi-stage build lets a SINGLE Dockerfile use MULTIPLE `FROM` stages — typically one stage with the FULL build toolchain (Maven, JDK) to COMPILE your application, and a SEPARATE, much smaller FINAL stage that only contains the RUNTIME needed to actually RUN it — dramatically shrinking the final production image size.

2. The Problem Without Multi-Stage Builds

```
# SINGLE-STAGE (BAD for production - includes the ENTIRE Maven + JDK build toolchain
# in your FINAL image, even though you only need it to COMPILE, not to RUN!)
FROM maven:3.9-eclipse-temurin-17
WORKDIR /app
COPY . .
RUN mvn clean package
ENTRYPOINT ["java", "-jar", "target/app.jar"]
# Final image size: potentially 600-800+ MB, carrying Maven/build tools unnecessarily
```

3. The Fix — Multi-Stage Build

```
# STAGE 1: "build" - has the FULL Maven+JDK toolchain, used ONLY to compile
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline # cache dependencies in their OWN layer
COPY src ./src
RUN mvn clean package -DskipTests

# STAGE 2: "final" - starts COMPLETELY FRESH, with just a minimal JRE
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar # copy ONLY the compiled jar from the "build" stage
ENTRYPOINT ["java", "-jar", "app.jar"]
# Final image size: often 150-200 MB - the ENTIRE Maven/JDK build toolchain never even
# exists in the final image at all!
```

4. Real Life Analogy

- **Restaurant**: A multi-stage build is like a restaurant's SEPARATE, industrial-sized prep kitchen (Stage 1: full of heavy-duty mixers, ovens, and bulk ingredients — everything needed to PREPARE a dish) versus the small, elegant PLATING station actually visible to customers (Stage 2: just the finished dish, ready to serve) — the customer-facing area never needs to carry around all that heavy prep equipment; only the FINISHED RESULT makes it to the final presentation.

5. Edge Cases

Case	Result
Forgetting `--from=build` on the final `COPY`	Docker looks for the file in the CURRENT (final) stage instead, which doesn't have it — build fails
Naming stages (`AS build`) vs using numeric indices (`--from=0`)	Both work; NAMED stages are far more readable, especially with 3+ stages

6. Common Mistakes

- Shipping the FULL build toolchain (Maven, npm, compilers) in the final production image — unnecessarily bloats image size and expands the attack surface (more installed software = more potential vulnerabilities).
- Not caching the dependency-download step in its OWN layer (as shown with `mvn dependency:go-offline`), causing dependencies to re-download on EVERY build even when only source code changed.

7. Frequently Asked Interview Questions

1. **What is a multi-stage Docker build?** A Dockerfile with multiple `FROM` stages — a heavier BUILD stage compiles the application, and a separate, minimal FINAL stage contains only what's needed to RUN it, copying just the compiled artifact between stages.
2. **Why does this matter for production?** It dramatically reduces final image size (faster deployment, less storage) and REDUCES THE ATTACK SURFACE, since build tools (compilers, package managers) aren't present in the actual running production image at all.
3. **What Dockerfile syntax copies a file from an earlier stage into a later one?** `COPY --from=<stage-name> <source> <destination>`.

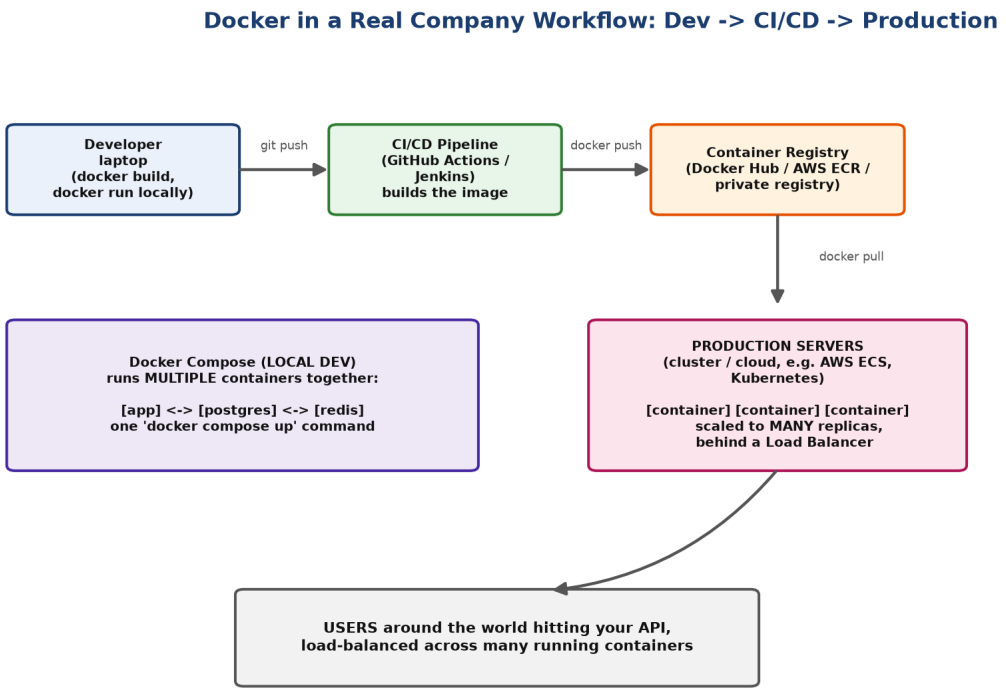
TOPIC 8: DOCKER AT COMPANY & PRODUCTION SCALE

---

1. Introduction

This topic ties everything together, addressing HOW Docker is actually used inside a real company — from a developer's first commit through to serving millions of users in production.

2. The Full Company Workflow



Docker in a real company workflow: Dev laptop to CI/CD to production, with Compose for local dev

```
1. DEVELOPER writes code, tests locally with `docker compose up` (Topic 4) - a full,
realistic environment (app + database + cache) running identically to production,
right on their own laptop.
2. Code is pushed to Git (previous section) -> triggers a CI/CD PIPELINE (GitHub Actions,
Jenkins - covered in the CI/CD section later in this course).
3. The CI pipeline runs: `docker build` -> `docker push` to a CONTAINER REGISTRY
(Docker Hub, AWS ECR, Google Artifact Registry, or a private company registry).
4. PRODUCTION SERVERS (often managed by an ORCHESTRATOR like Kubernetes or AWS ECS)
`docker pull` the newly-built image and start NEW containers running it.
5. A LOAD BALANCER distributes real user traffic across MANY running container
replicas of the SAME image, for both performance and fault tolerance.
```

3. Why Companies Use a Private Registry (Not Just Public Docker Hub)

```
# Tagging an image for a PRIVATE registry (e.g., AWS ECR)
docker tag employee-api:1.0.0 123456789.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.0.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.0.0
```

Company code/images are usually PROPRIETARY — a private registry keeps them secure and access-controlled, only pullable by authorized internal systems/team members, unlike Docker Hub's public repositories.

4. How Production Scales Docker — Orchestration (A Preview)

Running individual `docker run` commands doesn't scale to handling REAL production traffic reliably. Companies use an **orchestrator** — most commonly **Kubernetes**, or simpler options like **AWS ECS** — to automatically:
- Run MANY replicas (copies) of your container, spread across multiple physical servers.
- AUTOMATICALLY restart a container if it crashes.
- SCALE UP (add more replicas) automatically under high traffic, and scale down when traffic drops.
- ROLL OUT new versions gradually (e.g., "rolling updates" — replacing old containers with new ones a few at a time, so the application NEVER goes fully offline during a deployment).
- ROUTE traffic only to HEALTHY containers (via health checks), automatically removing unhealthy ones from rotation.

> Kubernetes itself is a large enough topic to warrant deep, dedicated study beyond this course's scope — but understanding that Docker CONTAINERS are the building block, and an ORCHESTRATOR manages them AT SCALE, is the key conceptual bridge between "I can run one container" and "this powers a production system handling millions of requests."

5. Health Checks — How Production Knows a Container Is Actually Working

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
CMD curl -f http://localhost:8080/actuator/health || exit 1
```

This ties DIRECTLY back to Spring Boot Actuator-style health endpoints — the orchestrator PERIODICALLY checks this, and automatically restarts or removes from rotation any container that starts failing its health check, WITHOUT requiring a human to notice and intervene manually.

6. Real Life Analogy

- **Restaurant Chain**: A single container is like ONE cook station running ONE specific dish. Production-scale Docker usage is like a MASSIVE, multi-location restaurant EMPIRE where a central management system (the orchestrator) automatically opens NEW cook stations during a dinner rush (scaling up), closes some during a slow afternoon (scaling down), instantly replaces a malfunctioning station (auto-restart on crash), and gradually rolls out a NEW recipe version across all locations WITHOUT ever fully shutting down service anywhere (rolling updates) — all coordinated centrally, without a human manually managing every single station by hand.

7. Common Mistakes

- Running raw, individual `docker run` commands directly on a production server manually — brittle, doesn't auto-restart on crash, doesn't scale; real production deployments use an orchestrator.
- Storing SECRETS (database passwords, API keys) directly baked into a Docker image — use environment variables injected at RUNTIME (or a dedicated secrets manager) instead, never baked into the image itself, which could be pulled/inspected by anyone with registry access.
- Running containers as the `root` user unnecessarily — a security best practice is to run as a NON-ROOT user inside the container wherever possible.

8. Best Practices

- Use multi-stage builds (Topic 7) for small, secure production images.
- Use a private, access-controlled registry for proprietary company images.
- Define health checks so orchestrators can automatically detect and recover from unhealthy containers.
- Never bake secrets into images — inject them at runtime via environment variables or a secrets manager.
- Use an orchestrator (Kubernetes, ECS) for any real production deployment beyond a single simple server.

9. Frequently Asked Interview Questions

1. **Walk through how a code change goes from a developer's laptop to production using Docker.** Developer tests locally (often with Compose) → pushes to Git → CI/CD builds and pushes a versioned image to a registry → production servers/orchestrator pull that image and run new containers → a load balancer routes traffic to the new, healthy containers.
2. **Why do companies use a private container registry instead of just public Docker Hub?** To keep proprietary code/images secure and access-controlled, pullable only by authorized internal systems and team members.
3. **What role does an orchestrator (like Kubernetes) play that plain `docker run` commands don't provide?** Automatic scaling, automatic restart on crash, rolling updates without downtime, and health-check-based traffic routing across many container replicas on many servers.
4. **How should secrets (passwords, API keys) be provided to a containerized application in production?** Injected at RUNTIME via environment variables or a dedicated secrets manager — never baked directly into the Docker image itself.
5. **What does a Docker `HEALTHCHECK` enable in a production/orchestrated environment?** The orchestrator can automatically detect an unhealthy container and restart it or remove it from load-balancer rotation, without requiring manual human intervention.

10. Revision Notes (Entire Docker Section Consolidated)

```
DOCKER — CHEAT SHEET
========================
SOLVES: "works on my machine" - packages app + EXACT environment, runs
identically everywhere
CONTAINER vs VM: containers SHARE the host OS kernel (MB-sized, starts in
seconds) vs VMs (full separate guest OS each, GB-sized, minutes to boot)

IMAGES: read-only templates. docker pull/build/images/rmi
ALWAYS pin exact version tags in production, never "latest"
CONTAINERS: running instances of images. docker run/ps/stop/start/rm/logs/exec
data is EPHEMERAL by default - lost on removal unless using a VOLUME

DOCKERFILE: FROM/WORKDIR/COPY/RUN/EXPOSE/ENV/ENTRYPOINT - each instruction =
a cached LAYER. Order least-changing -> most-changing for cache efficiency
DOCKER COMPOSE: docker-compose.yml + `docker compose up -d` runs MULTIPLE
coordinated containers together. Services reach each other by NAME.
depends_on only waits for START, not READINESS - use healthchecks

VOLUMES: persist data beyond a container's lifecycle
named volume (Docker-managed, portable) vs bind mount (exact host path, dev use)
NETWORKS: bridge (default, name-based DNS between containers) / host / none

MULTI-STAGE BUILDS: separate BUILD stage (full toolchain) from FINAL stage
(minimal runtime only) via multiple FROM + COPY --from=<stage> - smaller,
more secure production images

PRODUCTION/COMPANY SCALE: private registry (security/access control) ->
orchestrator (Kubernetes/ECS) for auto-scaling, auto-restart, rolling
updates, health-check-based routing. NEVER bake secrets into images -
inject at runtime.
```

11. Homework (Covers Entire Docker Section)

1. Install Docker Desktop, run `docker run hello-world`, then containerize a simple Spring Boot app using the Dockerfile from Topic 3 and run it locally.
2. Write a `docker-compose.yml` running your app alongside a PostgreSQL database, verify they can reach each other by service name, and confirm data survives a `docker compose restart` but is lost after `docker compose down -v`.
3. Convert your single-stage Dockerfile into a multi-stage build, and compare the final image sizes using `docker images`.
4. Research and write 3-4 sentences on the difference between how you'd deploy a container manually on one server versus how Kubernetes would manage the same container across many servers.

FINAL SUMMARY — DOCKER COMPLETE

This concludes the **Docker** section — 8 topics (6 core + 2 bonus, with diagrams):

1. **Images** — the "it works on my machine" problem solved, images vs VMs.
2. **Containers** — full local installation steps, the complete container lifecycle.
3. **Dockerfile** — full step-by-step Spring Boot containerization, layer caching.

4. **Docker Compose** — full multi-container local dev stack (app + Postgres + Redis).
5. **Volumes** — persisting data beyond a container's lifecycle.
6. **Networks** — how containers discover and reach each other by name.
7. **Multi-stage Builds** — small, secure production images.
8. **Docker at Company & Production Scale** — the full dev-to-production pipeline, private registries, orchestration, health checks, and secrets management.

You now understand Docker from "never touched it before" all the way through how it's actually used to run real production systems at scale — the direct foundation for the Kafka, Microservices, and AWS sections ahead, where containers are the fundamental deployment unit.

(END OF DOCKER MASTER NOTES — ready to proceed to Redis whenever you say "Next Topic.")